GFE 307

INTRODUCTION TO ARTIFICIAL INTELLIGENCE

Complete Simplified Study Edition

How the course fits together



Prepared from the supplied 71-page course material.
The explanations have been reorganised and rewritten in clearer language. The original course scope is retained, with diagrams redrawn to make important processes easier to understand.

For learning and revision • Engineering examples included

How to use this simplified edition

Read each topic in order the first time. Focus on the meaning of each term before memorising its definition. Use the diagrams to follow processes, then try to explain each process in your own words. The examples are intended to connect abstract AI ideas to engineering situations.

Source boundary: This edition is based on the supplied GFE 307 PDF. Where the source contains damaged dates, missing symbols, incomplete code, or unclear claims, this edition avoids guessing. Such issues are identified in the final source-quality notes.

Course coverage

- Human intelligence, artificial intelligence, computational intelligence and soft computing.
- Computational intelligence paradigms: fuzzy systems, neural networks, evolutionary computation, swarm intelligence, Bayesian networks and artificial immune systems.
- Search algorithms, decision trees, ensemble learning, supervised, unsupervised and reinforcement learning.
- Nature-inspired optimisation and engineering applications.
- Natural language understanding and natural language processing.
- Knowledge representation and knowledge elicitation.
- Logical-mathematical intelligence, mathematical models and foundations of AI.
- Expert systems, automated reasoning and pattern recognition.
- Distributed systems and the role of AI in them.
- Data and information security, intelligent web technologies and technological convergence.
- Neural networks, deep learning and Python libraries used in AI.

1. Human intelligence and artificial intelligence

1.1 Human intelligence

Human intelligence is the ability to learn, understand, reason, solve problems, use language, recognise patterns and adapt to new situations. Human beings also use experience, context, emotions, social understanding and creativity when making decisions.

People are particularly strong when a task requires empathy, moral judgement, flexible common sense or an understanding of subtle social situations. These abilities depend on more than calculating an answer: they involve interpreting the situation and considering how people may feel.

1.2 Artificial intelligence

Artificial intelligence (AI) is the field of designing computer systems that perform tasks normally associated with intelligent behaviour. Examples include recognising speech, identifying objects in images, translating languages, recommending actions, planning routes and detecting unusual patterns in data.

An AI system does not automatically understand the world as a human does. It processes input using programmed rules, learned patterns, mathematical models or a combination of these. Its performance depends on the task, the quality of its data and the way it was designed.

Human intelligence	Artificial intelligence
Learns through experience, education and interaction with the world.	May learn from data, feedback or rules supplied by people.
Can use broad context, empathy and common sense.	Usually performs best within a defined task or operating range.

Human intelligence	Artificial intelligence
Can adapt flexibly to unfamiliar situations, though humans also make errors.	Can process large volumes of information quickly but may fail outside its training or design conditions.
Uses biological processes in the brain and body.	Uses computer hardware, algorithms and data.

Engineering example: An engineer can diagnose a machine fault by combining measurements with experience, sound, smell and knowledge of the operating environment. An AI fault-detection system may compare sensor readings with patterns learned from previous faults. The system can assist the engineer, but its output still needs to be interpreted.

1.3 AI, computational intelligence and soft computing

The source discusses AI and computational intelligence (CI) as related but not always identical fields. AI is often used as a broad label for computer techniques that imitate or support intelligent behaviour. CI commonly focuses on adaptive methods that learn from data and tolerate uncertainty or imprecision.

Soft computing accepts that some real-world problems do not have perfectly precise inputs or exact solutions. It uses approximate reasoning, learning and optimisation to find useful answers. Hard computing generally expects clearly defined inputs and precise procedures.

Hard computing	Soft computing
Prefers exact inputs and clearly specified rules.	Can work with uncertainty, noisy data and approximate values.
Often uses conventional deterministic algorithms.	Often uses fuzzy logic, neural networks and evolutionary methods.
Useful when exactness and strict constraints are essential.	Useful when real-world complexity makes an exact model difficult.

1.4 How CI relates to AI

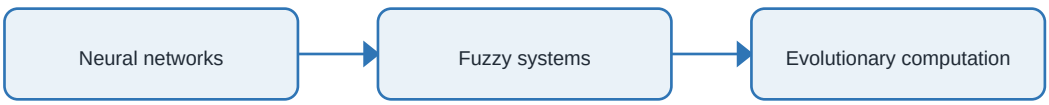
The course material describes several ways researchers view the relationship: CI may be treated as an alternative to AI, as a part of AI, or as a field that includes some AI methods. The important point for this course is that the fields overlap, and their boundaries are not universally described in one way.

Remember: Do not assume that AI and CI are always synonyms. Explain the relationship as an area of overlap whose exact boundary depends on the definition being used.

2. Computational intelligence paradigms

Computational intelligence uses adaptive computational methods to solve difficult problems. The course identifies three traditional pillars: neural networks, fuzzy systems and evolutionary computation. It also discusses swarm intelligence, Bayesian networks, artificial immune systems and learning theory.

Three traditional pillars of computational intelligence

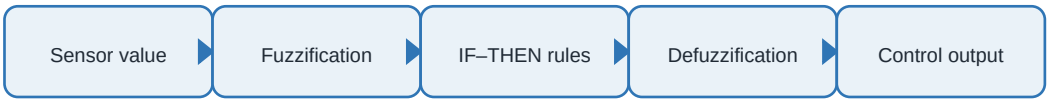


2.1 Fuzzy systems and fuzzy logic

Traditional Boolean logic assigns a statement one of two values: true or false. Fuzzy logic allows a statement to have a degree of truth between 0 and 1. This is useful when categories have gradual boundaries, such as 'slightly hot', 'warm' or 'very hot'.

A fuzzy controller commonly follows four stages: convert a measured value into fuzzy membership values (fuzzification); apply IF–THEN rules; combine the rule results; and convert the fuzzy result into a usable numerical output (defuzzification).

Basic fuzzy-control process

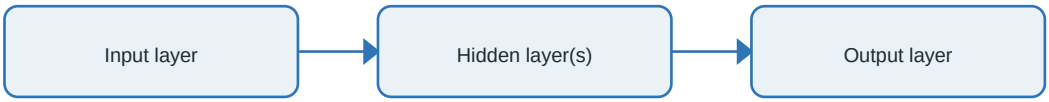


Example: A fan controller may use rules such as: IF temperature is high, THEN fan speed is fast. Because 'high' is not a perfectly sharp boundary, the fan can increase speed gradually rather than switching abruptly.

2.2 Neural networks

An artificial neural network is a computational model made of connected processing units called neurons. Each connection has a weight. A neuron combines its inputs, applies a mathematical activation function and passes its output to the next layer.

A simple feed-forward neural network



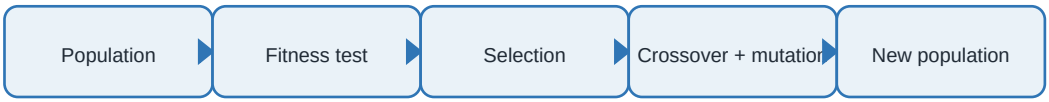
During training, the network compares its prediction with the expected answer, calculates an error or loss, and adjusts its weights to reduce that error. Repeating this process helps the network learn patterns in examples.

2.3 Evolutionary computation

Evolutionary computation is inspired by biological evolution. Instead of improving only one candidate solution, an algorithm works with a population of possible solutions. Better candidates are more likely to be selected to produce new candidates.

- **Initialisation:** create a population of candidate solutions.
- **Evaluation:** calculate how well each candidate solves the problem using a fitness function.
- **Selection:** choose candidates to reproduce.
- **Crossover:** combine parts of two candidate solutions.
- **Mutation:** make small random changes to maintain variation.
- **Replacement:** form the next population and repeat until a stopping condition is reached.

Evolutionary algorithm cycle



An engineering example is selecting dimensions of a mechanical component to reduce weight while meeting strength constraints. The fitness function evaluates each proposed design.

2.4 Swarm intelligence

Swarm intelligence takes inspiration from the collective behaviour of groups such as ants, bees, birds or fish. Individual agents follow relatively simple rules, but their interactions can produce useful group behaviour.

Ant colony optimisation models how ants deposit and follow pheromone trails. Candidate routes that perform well can receive stronger trail influence, making them more likely to be explored again. Particle swarm optimisation models particles moving through a search space while using their own best-known position and the group's best-known position.

2.5 Bayesian networks

A Bayesian network is a probabilistic graphical model. It represents variables as nodes and conditional relationships as directed edges. The graph must be acyclic: following the arrows must never lead back to the starting node.

The model helps calculate how likely an event is when new evidence becomes available. For example, sensor evidence may change the estimated probability that a motor is overheating.

Illustrative Bayesian dependency



2.6 Artificial immune systems

Artificial immune systems borrow ideas from the biological immune system, especially recognition, adaptation, memory and the selection of responses. In computing, these ideas can be used for anomaly detection, clustering and optimisation. A system may learn what normal patterns look like and flag unusual patterns for inspection.

2.7 Learning theory and probabilistic methods

Learning theory studies how a system can learn from examples and how well its learned behaviour may generalise to new cases. Probabilistic methods represent uncertainty using probabilities. Bayesian reasoning updates beliefs when evidence is received.

Key distinction: A neural network learns weighted relationships; fuzzy logic represents degrees of membership or truth; evolutionary computation searches among candidate solutions; swarm methods coordinate simple agents; Bayesian networks reason about probabilistic dependencies.

3. Search algorithms in AI

A search algorithm explores possible states or routes to find a goal. A search problem usually has an initial state, possible actions, a transition model, a goal test and sometimes a path-cost function.

For a route-finding problem, the initial state is the starting location, actions move to neighbouring locations, the goal test checks whether the destination has been reached, and path cost may represent distance, time or fuel.

3.1 Uninformed search

Uninformed, or blind, search does not use a heuristic estimate of how close a state is to the goal. It chooses what to explore using general rules such as depth or accumulated path cost.

Method	How it explores	Strengths and limitations
Depth-first search (DFS)	Follows one branch as deeply as possible, then backtracks.	Can use less memory; may miss a shorter solution and can follow a deep or cyclic path without safeguards.
Breadth-first search (BFS)	Explores all nodes at one depth before moving deeper.	Finds a shallowest solution; can require substantial memory.
Uniform-cost search (UCS)	Expands the frontier node with the lowest accumulated path cost g(n).	Finds a least-cost solution when step costs are non-negative; may explore many nodes.

Breadth-first search explores level by level



DFS is commonly implemented with a stack or recursion. BFS uses a queue. UCS uses a priority queue ordered by cumulative cost.

3.2 Informed search

Informed search uses additional knowledge called a heuristic. A heuristic h(n) estimates the remaining cost from node n to the goal. A good heuristic can guide the algorithm toward promising states and reduce unnecessary exploration.

Method	Evaluation	Meaning
Greedy best-first search	h(n)	Chooses the node that appears closest to the goal; it ignores the cost already paid.
A* search	f(n) = g(n) + h(n)	Balances the cost already travelled with the estimated cost remaining.

Here, g(n) is the actual cost from the start to node n; h(n) is the estimated cost from n to the goal; and f(n) is the estimated total route cost through n.

Simple example: A delivery robot has travelled 4 cost units to reach a junction. The estimated remaining cost to the destination is 6. A* assigns f = 4 + 6 = 10 to that route.

3.3 A* tree search and graph search

A* tree search treats each route as a separate path and does not maintain a record of all previously explored states. A* graph search keeps track of visited or explored states to avoid repeatedly processing the same state. Correct graph-search implementations must manage the possibility of finding a cheaper route to a state already encountered.

3.4 Comparing search methods

Algorithm	Uses heuristic?	Typical data structure	Important point
DFS	No	Stack	Memory-efficient in many cases; not generally optimal.
BFS	No	Queue	Optimal when every step has equal cost.
UCS	No	Priority queue by g(n)	Chooses lowest accumulated cost.
Greedy	Yes	Priority queue by h(n)	Can be fast, but may choose a costly route.
A*	Yes	Priority queue by g(n)+h(n)	Can be optimal with suitable conditions and heuristic.

3.5 Decision trees

A decision tree is a model that makes a prediction by following a sequence of questions. Internal nodes test a feature, branches represent possible outcomes and leaf nodes give a class or numerical prediction.

Decision-tree structure



For example, a maintenance decision tree might ask whether motor vibration exceeds a threshold. If yes, it may check temperature; the final leaf may recommend inspection or continued operation.

The course names Iterative Dichotomiser (ID3) and Classification and Regression Trees (CART). ID3 selects splits using information gain, which is based on entropy. CART builds binary splits and can be used for classification or regression.

3.6 Ensemble learning

Ensemble learning combines multiple models to produce a stronger overall prediction. The idea is that a group of models may perform better than one model if their errors are not all the same.

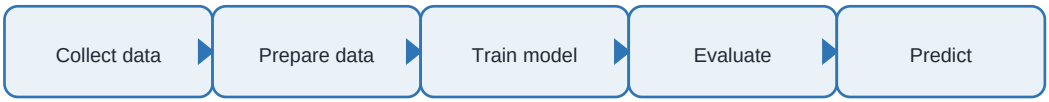
- **Bagging:** trains models on different resampled versions of the training data and combines their outputs. Random forests are a common example.
- **Boosting:** trains models in sequence, with later models focusing more on examples earlier models handled poorly. AdaBoost is one example.
- **Stacking:** combines predictions from several base models using another model, called a meta-model, to learn how to combine them.

Important: An ensemble is not automatically better. It depends on data quality, model diversity, training design and evaluation.

4. Machine learning

Machine learning (ML) is a branch of AI in which a computer system learns patterns from data and uses those patterns to make predictions or decisions. A typical workflow is to collect data, prepare it, train a model, evaluate it and then use it on new data.

A basic machine-learning workflow



4.1 Supervised learning

Supervised learning uses examples that include both input features and known target answers (labels). The model learns a relationship between the inputs and the target.

- **Classification:** predicts a category, such as whether a component is faulty or healthy.
- **Regression:** predicts a numerical value, such as the expected remaining life of a bearing.

A decision tree, support vector machine, linear regression model or neural network can be used for supervised tasks.

4.2 Unsupervised learning

Unsupervised learning uses data without supplied target labels. The system looks for structure, groups, relationships or unusual observations.

Clustering methods described in the course include K-means, K-means++, K-modes, fuzzy C-means, distribution-based methods such as expectation-maximisation and Dirichlet process mixture models, and connectivity-based methods such as agglomerative clustering.

Method	Basic idea	Typical use
K-means	Groups numerical data around k cluster centres.	Grouping sensor readings or customer records.
K-means++	Chooses initial centres more carefully to improve K-means initialisation.	More stable cluster starting points.
K-modes	Adapts clustering to categorical data.	Grouping records with non-numeric categories.
Fuzzy C-means	Allows an item to belong to multiple clusters with different membership degrees.	Overlapping operating conditions.
Expectation-maximisation	Alternates between estimating hidden memberships and updating model parameters.	Mixture-model clustering.
Agglomerative clustering	Starts with individual items and repeatedly merges similar groups.	Hierarchical grouping.

4.3 Reinforcement learning

Reinforcement learning (RL) involves an agent interacting with an environment. The agent observes a state, chooses an action, receives a reward or penalty and moves to a new state. Over time, it learns a policy that aims to maximise long-term reward.

Reinforcement-learning feedback loop



A robot learning to move through a room may receive positive reward for reaching a target and penalties for collisions. It gradually learns which actions produce better long-term outcomes.

4.4 Markov decision processes

A Markov decision process (MDP) is a mathematical framework for decision-making when outcomes are partly uncertain. It is commonly described using states, actions, transition probabilities, rewards and a discount factor.

The Markov property means that the next state depends on the current state and action, rather than requiring the full history of earlier states. This is a modelling assumption, not a claim that history never matters in real systems.

4.5 Bellman equation and value iteration

The Bellman equation expresses the value of a state in terms of the immediate reward and the expected value of future states. A common optimal-value form is:

$$V^*(s) = \max_a \Sigma_{s'} P(s'|s,a) \, [\, R(s,a,s') + \gamma \, V^*(s') \,]$$

Here, $V^*(s)$ is the best expected long-term value of state s; a is an action; $P(s'|s,a)$ is the probability of reaching next state s'; R is the reward; and γ is the discount factor between 0 and 1.

Value iteration repeatedly updates state values using the Bellman relationship until the values change very little. A policy can then be selected by choosing the action with the highest expected value.

4.6 Monte Carlo tree search

Monte Carlo Tree Search (MCTS) explores possible future actions by repeatedly running simulations. Its common stages are selection, expansion, simulation and backpropagation. It is useful when the search space is large and exhaustive search is impractical.

4.7 Q-learning and SARSA

Q-learning is an off-policy reinforcement-learning method. It updates an estimate $Q(s,a)$ of the value of taking action a in state s, using the best estimated next action value.

$$Q(s,a) \leftarrow Q(s,a) + \alpha \, [\, r + \gamma \max_{a'} Q(s',a') - Q(s,a) \,]$$

SARSA is an on-policy method. Its update uses the next action actually selected by the current policy:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \, [\, r + \gamma \, Q(s',a') - Q(s,a) \,]$$

The name SARSA comes from the sequence State, Action, Reward, State, Action. The distinction is that Q-learning uses the maximum next-action value, while SARSA uses the value of the action the policy actually chooses.

4.8 Monte Carlo methods and REINFORCE

Monte Carlo methods learn from complete sampled episodes or returns. They estimate values using observed outcomes rather than requiring a complete model of the environment. REINFORCE is a policy-gradient method: it adjusts a policy's parameters using sampled returns to make rewarding actions more likely.

Exam focus: Be able to distinguish supervised, unsupervised and reinforcement learning; explain the agent–environment loop; and state the difference between Q-learning and SARSA.

5. Nature-inspired optimisation algorithms

Nature-inspired optimisation methods imitate processes observed in nature to search for good solutions. They are especially useful when a problem has many possible solutions, a complicated objective function or no simple exact solution.

5.1 General optimisation problem

An optimisation problem asks us to minimise or maximise an objective while satisfying constraints. For example, an engineer may minimise the weight of a structure while ensuring that stress remains below a safe limit.

Minimise or maximise: f(x)
Subject to: g■(x) ≤ 0 and h■(x) = 0

5.2 Common nature-inspired methods

- **Genetic algorithms:** evolve a population of candidate solutions using selection, crossover and mutation.
- **Particle swarm optimisation:** moves a population of particles through a search space using individual and group experience.
- **Ant colony optimisation:** uses artificial pheromone trails to guide route or combinatorial search.
- **Artificial immune algorithms:** use ideas such as recognition, memory and selection.
- **Other swarm methods:** model collective search behaviour inspired by social insects, birds or other organisms.

5.3 Engineering applications

Application area	Example of the optimisation task
Digital filter design	Choose filter coefficients to meet frequency-response requirements while controlling error or complexity.
Image processing	Select parameters that improve image quality, segmentation or feature extraction.
Machine learning	Tune model parameters or select features to improve predictive performance.
Power and energy systems	Optimise scheduling, generation or energy use subject to operating constraints.
Mechanical design	Find dimensions or materials that balance strength, cost and weight.

Practical caution: Nature-inspired methods do not guarantee the global best solution in every run. Results depend on parameter settings, search space, stopping rules and repeated evaluation.

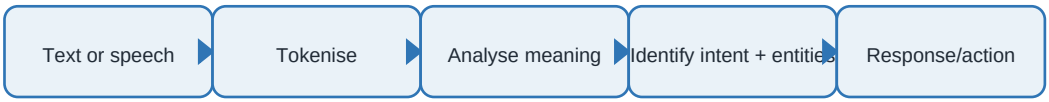
6. Understanding natural language

6.1 Natural language understanding

Natural Language Understanding (NLU) is the part of language technology concerned with interpreting the meaning and intention behind human language. It aims to identify what a person means, not merely recognise the words.

For example, a student might type, 'Show me the courses I need this semester.' An NLU system may identify the intent as requesting a course list and extract the semester as a relevant detail.

Simplified NLU pipeline



6.2 Steps in NLU

- **Input processing:** receive text or convert speech into text.
- **Tokenisation:** split text into words or smaller units.
- **Syntax analysis:** examine grammatical structure and relationships.
- **Semantic analysis:** determine likely meaning using the words and their relationships.
- **Intent and entity extraction:** identify what the user wants and important details such as dates, locations or quantities.
- **Context handling:** use relevant conversation context to interpret references and ambiguity.
- **Action or response:** provide an answer or pass the interpreted request to another system.

6.3 Limitations of NLU

Language is ambiguous. The same word can have several meanings, and a sentence may depend on context not stated directly. Sarcasm, irony, humour, slang, dialects, spelling errors and cultural references can also be difficult to interpret reliably.

Example: 'That machine is really fast' may be praise. 'That machine is really fast… it broke down again' may be sarcastic. The literal words alone are not enough to identify the speaker's intention.

6.4 Natural language processing

Natural Language Processing (NLP) is the broader field of enabling computers to process human language. It includes tasks such as language detection, translation, speech recognition, text classification, information extraction, summarisation and text generation.

Term	Main purpose	Example
NLP	Process and work with human language.	Translate a paragraph or classify an email.
NLU	Interpret meaning and intent.	Understand that 'book a table' is a reservation request.
NLG	Generate human-readable language.	Produce a written answer or report.

6.5 Common NLP techniques and technologies

- **Tokenisation:** split text into words, subwords or sentences.
- **Stemming and lemmatisation:** reduce words to a root or dictionary form.
- **Part-of-speech tagging:** identify nouns, verbs, adjectives and other grammatical roles.
- **Named-entity recognition:** detect names of people, organisations, places, dates and similar entities.
- **Parsing:** analyse grammatical structure.
- **Sentiment analysis:** estimate the opinion or emotional tone expressed in text.
- **Machine translation:** convert text from one language to another.

- **Speech technologies:** convert speech to text or generated text to speech.
- **Information retrieval:** find relevant documents or passages from a collection.

6.6 NLP applications

- Search engines and question-answering systems.
- Chatbots and virtual assistants.
- Spam detection and email filtering.
- Translation and transcription.
- Sentiment analysis of reviews and feedback.
- Document classification, summarisation and information extraction.

7. Knowledge representation in AI

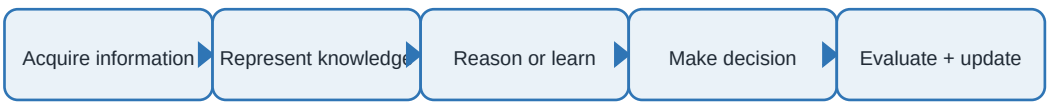
Knowledge representation is the process of organising information in a form a computer can use for reasoning, answering questions or making decisions. A system needs more than raw data: it needs a way to represent objects, properties, relationships, rules and constraints.

7.1 Main methods

Method	How it represents knowledge	Example
Propositional logic	Statements that are true or false, joined by logical operators.	'The alarm is active'.
First-order logic	Objects, properties, relations and quantified statements.	'Every motor has a rated voltage'.
Semantic networks	Concepts as nodes and relationships as links.	A motor IS-A machine.
Frames	Structured records with slots and values for a familiar object or situation.	A motor frame with power, speed and voltage slots.
Ontologies	Formal definitions of concepts and relationships in a domain.	A model of components, faults and maintenance actions.
Bayesian networks	Probabilistic relationships between variables.	Sensor evidence changes fault probability.
Markov decision processes	States, actions, transitions and rewards for sequential decisions.	A robot chooses its next movement.
Embeddings	Numerical vectors that represent items so similarity can be calculated.	Words or documents with related meanings may have nearby vectors.
Knowledge graphs	Entities and their relationships represented as a graph.	A component is part of a system and is supplied by a manufacturer.

7.2 The knowledge cycle

A simplified AI knowledge cycle



Knowledge may be acquired from experts, documents, databases, sensors or previous system experience. It is then represented in a suitable format and used by reasoning or learning processes. Results can reveal where the knowledge needs to be updated.

7.3 Types of knowledge

- **Declarative knowledge:** facts about objects, properties and events — knowing that something is true.
- **Procedural knowledge:** knowledge of how to perform a task or sequence of steps.
- **Heuristic knowledge:** practical rules of thumb that often work but are not guaranteed.
- **Meta-knowledge:** knowledge about other knowledge, such as when a rule should be used.
- **Structural knowledge:** knowledge about how concepts or components are related.

7.4 Challenges and applications

Knowledge can be incomplete, inconsistent, uncertain, outdated or difficult to express. A representation may also become expensive to maintain as the domain grows. AI uses knowledge representation in expert systems, recommendation systems, autonomous vehicles, virtual assistants, planning and decision support.

Engineering example: A maintenance knowledge graph could connect a pump, its bearings, common failure modes, sensor symptoms, inspection procedures and replacement parts.

8. Knowledge elicitation

Knowledge elicitation is the process of obtaining knowledge from people, records or other sources so it can be documented and used in an expert system or other knowledge-based application. It is especially important when experienced specialists hold knowledge that is not written down.

8.1 Interviews

An interview allows a knowledge engineer to ask an expert how they diagnose problems or make decisions. Interviews may be structured, semi-structured or open-ended. A structured interview uses prepared questions; a semi-structured interview allows follow-up questions; an open interview gives the expert more freedom to explain.

8.2 Other elicitation techniques

Technique	What it does
Protocol analysis	Asks an expert to think aloud while performing a task; the explanation reveals steps and decision cues.
Critical Decision Method	Examines a specific difficult or unusual event to discover what information and decisions mattered.
Concept sorting	Asks experts to group concepts and explain the basis for their groupings.
Repertory grids	Uses comparisons between examples to reveal the dimensions or attributes experts use.
Laddered grids	Explores relationships between categories and attributes in a structured way.
Limited Information Task	Gives the expert incomplete information and observes what they ask for or infer.
Concept mapping	Creates a visual map of concepts and their relationships.
Process mapping	Shows the stages, decisions and flows involved in a task.

8.3 Natural and contrived techniques

The course groups elicitation techniques into natural or non-contrived approaches and contrived approaches. Natural approaches observe or capture expertise in ordinary work settings. Contrived approaches create a structured task or controlled situation to reveal how the expert thinks.

Examples named in the course include fixed probes, forward scenario simulation and the Knowledge Acquisition Grid. The exact classification can depend on how a particular study is designed.

Good practice: Record the expert's assumptions, exceptions and confidence—not only the final answer. Check the resulting knowledge model with the expert before treating it as authoritative.

9. Logical-mathematical intelligence and mathematical models

Logical-mathematical intelligence refers to the ability to reason logically, recognise patterns, work with quantities and solve structured problems. In AI, mathematical and logical methods provide tools for representing problems, evaluating possibilities and selecting actions.

9.1 Key characteristics

- Logical reasoning: drawing conclusions from facts and rules.
- Pattern recognition: identifying repeated structures or relationships.
- Quantitative reasoning: using numbers, measurements and mathematical relationships.
- Problem decomposition: dividing a complex task into smaller manageable parts.
- Abstract reasoning: working with symbols, models and general rules.

9.2 Mathematical models in AI

A mathematical model is a simplified representation of a real system. It describes important variables and relationships using equations, functions, probabilities or constraints. Models allow engineers to analyse a system without testing every possibility in the physical world.

- **Optimisation models:** choose values that minimise cost, time, energy or error.
- **Probability models:** represent uncertainty and estimate the likelihood of events.
- **Statistical models:** identify relationships and patterns in observed data.
- **Logical models:** represent rules and derive conclusions.
- **Dynamic models:** describe how a system changes over time.

A model is only as useful as its assumptions and data. Engineers should compare model predictions with observed behaviour and revise the model when important differences appear.

9.3 Role in AI

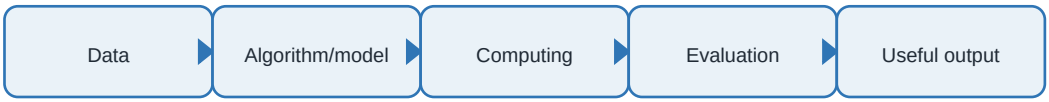
Mathematics supports machine learning through linear algebra, calculus, probability, statistics and optimisation. Logic supports symbolic reasoning. Discrete mathematics supports graphs, search and data structures. These foundations help explain how AI systems are trained and how they make decisions.

10. Foundations of artificial intelligence

10.1 Core concepts

AI combines data, algorithms, computing resources and evaluation methods to perform tasks that require perception, prediction, reasoning, language processing or decision-making. Some systems use explicit rules; others learn statistical patterns from examples; many combine both approaches.

Main ingredients of an AI application



10.2 How an AI system works

- Define the task and decide what counts as a successful result.
- Collect relevant data or encode the required expert rules.
- Prepare inputs so the system can process them.
- Choose or design an algorithm/model.
- Train, configure or execute the system.
- Evaluate its performance using suitable test cases.
- Deploy it, monitor its behaviour and update it when necessary.

10.3 Types of AI

The course introduces types of AI. A common capability-based distinction is narrow AI, which is designed for a limited task, and general AI, which would be able to perform a broad range of intellectual tasks at a human-like level. The source does not establish that general AI is an existing, achieved capability.

Type	Meaning	Example or status
Narrow AI	Designed for a defined task or limited range of tasks.	Image classifier, spam filter, route planner.
General AI	Hypothetical system with broad, flexible intellectual abilities across many domains.	A research goal; not an established everyday system.

10.4 Advantages and limitations

- Can process large volumes of data quickly.
- Can automate repetitive or time-consuming tasks.
- Can detect patterns that may be difficult to notice manually.
- Can support consistent decisions when rules and data are suitable.
- May produce biased or inaccurate results if its data or design is flawed.
- May be difficult to explain, maintain or secure.
- Can fail when operating conditions differ from those it was designed or trained for.

10.5 Applications

AI is used in manufacturing, predictive maintenance, medical support, transport, energy management, agriculture, education, finance, security and communication. In engineering, examples include fault diagnosis, quality inspection, demand forecasting, robot navigation and optimisation of system operation.

Responsible use: AI outputs should be validated, especially when decisions affect safety, health, money or critical infrastructure.

11. Expert systems

An expert system is an AI program designed to solve problems in a specific domain by using knowledge and reasoning rules supplied by human experts. It aims to reproduce parts of an expert's decision process, not the expert's entire human ability.

11.1 Main components

Architecture of a rule-based expert system



The knowledge base stores facts and rules. The inference engine applies rules to known facts to derive conclusions. The user interface allows a person to provide information and receive results. An explanation facility can show why the system reached a conclusion.

11.2 How it works

- The user provides facts about a case.
- The system stores or matches those facts against its knowledge base.
- The inference engine selects rules whose conditions are satisfied.
- The rules produce new facts, recommendations or questions.
- The process continues until a conclusion is reached or no further rule applies.
- The system presents the result and, where possible, explains its reasoning.

11.3 Reasoning strategies

Forward chaining is data-driven: start with known facts and apply rules to derive new facts until a conclusion or stopping condition is reached. **Backward chaining** is goal-driven: start with a possible conclusion and work backwards to determine whether the required supporting facts can be established.

Forward and backward chaining



11.4 Types and examples

Expert systems may be rule-based, frame-based, fuzzy, model-based or case-based, depending on how knowledge is represented and how conclusions are produced. The course discusses expert systems in areas such as diagnosis, configuration, monitoring, planning and decision support.

11.5 Applications and benefits

- Equipment fault diagnosis and troubleshooting.
- Medical decision support.

- Industrial process monitoring.
- Configuration of complex systems.
- Planning and scheduling.
- Preserving and distributing specialist knowledge.

Benefits include consistent application of rules, fast access to stored expertise and support for less experienced users. Limitations include narrow scope, difficulty acquiring expert knowledge, maintenance costs and inability to handle cases outside the knowledge base reliably.

12. Automated reasoning

Automated reasoning is the use of computer methods to derive conclusions from facts, rules, assumptions or mathematical statements. Unlike purely data-driven prediction, it can work with explicit logical structures and formal relationships.

12.1 Logical frameworks and inference

A logical framework defines the language used to express statements and the rules that allow conclusions to be derived. Inference techniques include applying logical rules, checking consistency, proving theorems and deriving consequences from known facts.

12.2 Applications

- **Automated theorem proving:** checking whether a mathematical statement follows from given assumptions.
- **Software verification and testing:** checking whether a program satisfies specified properties.
- **Decision support:** applying rules and constraints to recommend an action.
- **Robotics:** planning sequences of actions while respecting rules and constraints.
- **Natural language understanding:** using structured meaning and logical relationships to support interpretation.
- **Autonomous vehicles:** reasoning about permitted actions, road rules and possible situations.
- **Simulation and optimisation:** using formal constraints to evaluate possible system states.

12.3 Why reasoning is challenging

Logical reasoning can become computationally expensive as the number of facts, rules and possible combinations grows. Real-world information may be incomplete, ambiguous or inconsistent. A reasoning system also needs a useful representation of the domain; without it, the rules cannot be applied effectively.

12.4 Data-driven AI and reasoning-based AI

Data-driven methods	Reasoning-based methods
Learn patterns from examples.	Derive conclusions from explicit facts and rules.
Can perform well when large, representative datasets exist.	Can be useful when rules and constraints are known but data is limited.
May be difficult to explain in detail.	Can often provide a trace of the rules used.
Can struggle with distribution changes or biased data.	Can struggle with incomplete knowledge and large search spaces.

The two approaches can be combined. A system may use machine learning to interpret sensor data and logical reasoning to check whether a proposed action follows safety rules.

12.5 Future direction and ethical issues

Automated reasoning continues to develop through improved solvers, verification methods and integration with machine learning. Responsible systems must also consider fairness, accountability, transparency, safety and the consequences of incorrect conclusions.

13. Types of reasoning in AI

Reasoning type	Meaning	Example
Deductive	Applies general rules to reach a logically necessary conclusion when premises are true.	All devices above a temperature limit trigger an alarm; this device is above the limit; therefore the alarm rule applies.
Inductive	Uses observations to form a general pattern or likely rule.	Several bearings fail after repeated overheating; overheating may be a risk indicator.
Abductive	Finds the most plausible explanation for an observation.	A motor vibrates and overheats; bearing damage is one possible explanation to investigate.
Analogical	Uses similarities with a known case to reason about a new case.	A new pump fault resembles a previously diagnosed pump fault.
Probabilistic	Uses probability to represent uncertainty.	Sensor evidence raises the estimated probability of a fault.

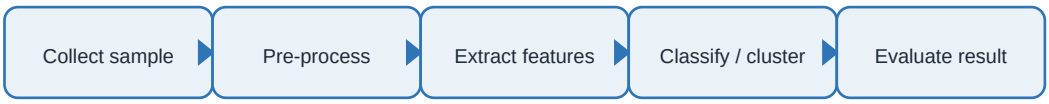
Do not confuse them: Deduction aims at a conclusion that follows from the premises. Induction generalises from observations. Abduction proposes a likely explanation that still needs checking.

14. Pattern recognition

Pattern recognition is the process of identifying regularities in data and assigning observations to categories or meaningful patterns. It is used with images, sound, text, sensor readings and other data.

14.1 Typical pattern-recognition pipeline

Pattern recognition process



Pre-processing may remove noise, correct scale or standardise the data. Feature extraction converts raw data into useful measurements. A classifier or clustering algorithm then identifies the pattern.

14.2 Training and learning

In supervised pattern recognition, examples are labelled so a model can learn to associate features with categories. In unsupervised pattern recognition, the system searches for groups or structure without supplied labels. Performance should be evaluated on data not used to train the model.

14.3 Example: identifying a machine condition

Suppose vibration data is collected from a rotating machine. Features might include average vibration, peak amplitude and frequency components. A model can learn to distinguish normal operation from patterns associated with imbalance or bearing wear. The model's output should be checked against reliable maintenance evidence.

14.4 Advantages and disadvantages

- **Advantages:** supports classification, detects patterns in large datasets, assists image and speech analysis, and can identify unusual biometric or machine patterns.
- **Disadvantages:** performance depends on data quality and representation; large datasets may be needed; noise and changing conditions can reduce accuracy; some methods are computationally complex and difficult to interpret.

14.5 Python implementation concept

A simple nearest-neighbour classifier compares a new sample with stored training samples. It calculates a distance, such as Euclidean distance, and assigns the new sample to the class of the closest example or the majority class among the k closest examples.

Euclidean distance: $d(x,y) = \sqrt{\Sigma\blacksquare\ (x\blacksquare - y\blacksquare)^2}$

Other distance measures mentioned in the course include Manhattan and Minkowski distance. Feature scaling matters: a feature measured in thousands can dominate one measured between 0 and 1 unless the data is appropriately scaled.

Source note: The source PDF includes a Python pattern-recognition example, but some code formatting is compressed. The explanation here preserves the stated nearest-distance idea without reproducing potentially corrupted code.

15. Distributed systems

A distributed system is a collection of independent computers or devices that communicate over a network and cooperate to provide a service. To a user, the system may appear to be one application even though its work is spread across several machines.

15.1 Centralised versus distributed systems

Centralised system	Distributed system
Most processing and data are controlled by one central computer or service.	Processing and data may be spread across multiple connected computers.
Can be simpler to manage in a small environment.	Can scale by adding machines and may continue operating when some components fail.
A central failure can stop the whole service.	Coordination, network and partial-failure problems must be handled.

15.2 Common architectures

Architecture	Description
Client-server	Clients request services; servers process requests and return responses.
Peer-to-peer (P2P)	Peers can both request and provide resources or services.
Service-oriented architecture (SOA)	Functions are provided as reusable services that communicate through defined interfaces.
Event-driven	Components react to events or messages rather than relying only on direct requests.

15.3 Characteristics

- Resource sharing across connected machines.
- Concurrency: several activities can occur at the same time.
- Scalability: capacity can be increased by adding resources.
- Fault tolerance: the system can continue despite some component failures.
- Transparency: users may not need to know where processing occurs.
- Communication and coordination between components.

15.4 Advantages, disadvantages and use cases

Distributed systems can improve scalability, resource sharing, availability and geographic reach. They also introduce complexity: networks can be slow or fail, data can become inconsistent, security boundaries multiply, and diagnosing faults can be difficult.

- Cloud computing and storage.
- Online banking and payment platforms.
- Content delivery networks.
- Distributed databases.
- Microservices-based applications.
- Collaborative tools and large-scale data processing.

15.5 AI in distributed systems

AI can help manage a distributed system by analysing performance data and making predictions or decisions. Examples include intelligent load balancing, resource allocation, anomaly detection, predictive maintenance, security monitoring and automated recovery.

AI technique	Possible role in a distributed system
Machine learning	Predict demand, identify patterns and classify system events.
Deep learning	Analyse complex or high-volume data, such as logs or network traffic.
Anomaly detection	Flag behaviour that differs from normal system activity.
Reinforcement learning	Learn resource-allocation or routing policies through feedback.
Natural language processing	Help users query system status or analyse incident reports.

15.6 AI-driven distributed architectures

- **AI-enhanced load balancing:** direct requests toward services with available capacity.
- **Intelligent resource management:** allocate CPU, memory, storage or network capacity based on predicted demand.
- **Fault detection and recovery:** identify abnormal behaviour and trigger a recovery procedure.
- **Enhanced security:** detect suspicious access patterns or unusual traffic.
- **Distributed data analytics:** process information across multiple machines and combine results.

15.7 Challenges

The course highlights data quality and management, scalability, latency and real-time processing, security and privacy, and model deployment and management. AI models themselves also need monitoring: a model that becomes inaccurate can make the distributed system less reliable.

Engineering example: A factory may use sensors on many machines. A distributed platform collects readings from several production areas, while AI models detect unusual vibration patterns and alert maintenance staff.

16. Data and information security in AI

AI systems depend on data for training, testing and operation. If data is stolen, altered, poisoned or exposed, the system and the people relying on it may be harmed. Data security protects information against unauthorised access, disclosure, modification or destruction.

16.1 Common threats

- **Data breaches:** unauthorised access to confidential datasets.
- **Data poisoning:** deliberately inserting misleading data into training or update processes.
- **Adversarial inputs:** inputs designed to cause a model to make an incorrect prediction.
- **Model theft:** copying or extracting a model or its behaviour.
- **Model inversion:** attempting to infer sensitive information about training data from model outputs.
- **Malware and ransomware:** malicious software that steals, damages or blocks access to data or systems.
- **Insider threats:** misuse of legitimate access by an employee or other authorised person.

16.2 Protecting AI systems

- Secure the data collection, storage, transfer and deletion processes.
- Restrict access according to user roles and the principle of least privilege.
- Encrypt sensitive information in storage and during transmission.
- Validate and monitor training data to detect suspicious changes.
- Keep software, libraries and infrastructure updated.
- Test models against malicious or unusual inputs.
- Maintain logs, backups and incident-response procedures.
- Monitor deployed models for unexpected behaviour or performance changes.

16.3 Privacy methods

Anonymisation removes or transforms identifying information so a person cannot reasonably be identified from the resulting data. **Pseudonymisation** replaces direct identifiers with codes or aliases; the information may still be re-linked if the additional key or information is available.

Before pseudonymisation	After pseudonymisation
Name: John Smith	Name: User-104
Age: 42	Age: 42
City: New York	City: New York
Medical condition: example condition	Medical condition: example condition

Important: Pseudonymised data is not automatically anonymous. Other details may still identify a person, so access controls and privacy safeguards remain necessary.

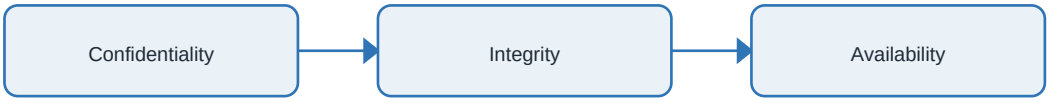
16.4 Best practices and ethical considerations

Organisations should collect only the data they need, explain how it will be used, protect it appropriately, limit retention, and respect applicable privacy requirements. The course names GDPR and the California Consumer Privacy Act (CCPA) as examples of data-protection legislation. Legal obligations depend on jurisdiction and context.

Ethical AI also considers fairness, transparency, accountability, human oversight and the consequences of errors. Security and privacy should be designed into the system rather than added only after deployment.

16.5 Information security and the CIA triad

The three goals of information security



- **Confidentiality:** only authorised people or systems can access information.
- **Integrity:** information remains accurate and is not changed improperly.
- **Availability:** authorised users can access information when required.

16.6 Types and uses of information security

The source discusses application security, data security, endpoint security and cloud security. It also describes authentication, encryption, network security, physical security, compliance, risk management and disaster recovery. Together, these controls protect systems, information and the services that depend on them.

16.7 Common information-security issues

- Cyber threats such as phishing, malware and ransomware.
- Human error, including weak passwords or unsafe links.
- Insider threats and excessive access privileges.
- Legacy systems with outdated security controls.
- Complex systems and third-party integrations.
- Mobile and IoT devices that may be lost, stolen or poorly secured.
- Privacy requirements across different regions and legal systems.

Security reminder: No single tool provides complete protection. Security requires layers of technical controls, sound procedures, user awareness and regular review.

17. Intelligent web technologies

The course describes Web 4.0 as an intelligent-web vision in which AI, machine learning and connected technologies make online services more adaptive and responsive. It imagines the web moving beyond displaying information toward systems that can interpret requests, personalise services and coordinate connected devices.

17.1 Features discussed in the course

- **Artificial intelligence:** systems analyse data and adapt services to user needs.
- **Natural language processing:** users interact with services using ordinary language.
- **Internet of Things (IoT):** connected devices exchange information between digital and physical environments.
- **Blockchain and decentralisation:** distributed records and applications may give users different ways to manage identity or transactions.
- **Augmented and virtual reality:** digital information or immersive environments are combined with user experiences.

17.2 Possible effects

Area	Possible effects described
Individuals	More personalised services, automated daily tasks and potential improvements in control over personal data.
Businesses	Data-driven decisions, collaboration tools and new digital services.
Society	New educational experiences, remote healthcare support and more efficient infrastructure.

17.3 Challenges

- Privacy risks from collecting large amounts of personal data.
- Digital divide: unequal access to devices, networks and skills.
- Cybersecurity risks from greater connectivity.
- Overdependence on automated systems.
- Complexity and cost of building and maintaining infrastructure.

Interpretation note: The source presents Web 4.0 as an evolving vision. Treat its features as proposed or developing capabilities, not as a guarantee that every listed technology is already universally deployed.

18. Convergent technologies

Technological convergence occurs when technologies that were once separate become integrated or combine to provide new services. Digital convergence is a related process in which different forms of information and media are handled through connected digital systems.

18.1 Examples

- **Mobile phones:** combine calling, messaging, internet access, cameras, media playback, navigation and computing functions.
- **Internet and media:** news, radio, television and social communication can be accessed through connected digital platforms.
- **Smart appliances:** household devices use sensors, software and networks to exchange information or respond automatically.
- **Telecommunications:** voice, video and data services can travel through shared network infrastructure.
- **Social and digital platforms:** content creation, publishing, communication and distribution operate together.

18.2 Converging technology fields

The source lists acronyms used to describe combinations of scientific and technological fields, including NBIC (nanotechnology, biotechnology, information technology and cognitive science), GNR (genetics, nanotechnology and robotics), GRIN (genetics, robotics, information and nano processes), GRAIN (genetics, robotics, artificial intelligence and nanotechnology), and BANG (bits, atoms, neurons and genes).

18.3 Significance and engineering applications

Convergence can create new products, improve efficiency and allow information to move between devices and services. In engineering, it appears in smart manufacturing, connected vehicles, sensor networks, robotics, telemedicine, smart homes and integrated energy systems.

Example: A smartphone is a convergent device because it combines functions once performed by separate products, including a telephone, camera, music player, map, web browser and computer.

19. Neural networks and deep learning

A neural network is a model made of connected artificial neurons arranged in layers. Deep learning refers to neural-network approaches that use multiple processing layers to learn increasingly complex representations from data.

Neural network (general term)	Deep learning
Can refer to networks with a range of architectures and depths.	Usually refers to neural networks with multiple layers and more complex learned representations.
May be relatively small and use fewer parameters.	Often uses larger models and substantial training data and computing resources.
Can be suitable for simpler or structured tasks.	Often useful for complex tasks such as image, speech and language processing.

19.1 Basic structure

Typical neural-network structure



The input layer receives features. Hidden layers transform those features using weighted connections and activation functions. The output layer produces a prediction, such as a class probability or numerical estimate.

19.2 Training in simple terms

- The network receives training examples.
- It produces a prediction.
- A loss function measures the difference between prediction and target.
- Backpropagation calculates how the weights contributed to the error.
- An optimiser updates the weights to reduce the loss.
- The process repeats over many examples and training iterations.

Deep learning can learn useful features directly from complex data, but it can require large datasets, substantial computing power and careful tuning. It may also be difficult to interpret.

Clarification: Deep learning is not a completely separate alternative to neural networks. It is a family of approaches built using neural networks with multiple layers.

20. Python and AI libraries

Python is widely used in AI because its syntax is relatively readable, it runs on multiple operating systems, it has a large developer community and it provides libraries for data handling, numerical computing, machine learning and deep learning.

20.1 Why Python is useful

- Readable syntax makes programs easier to write and maintain.
- A large ecosystem provides tools for common AI tasks.

- Libraries reduce the amount of low-level code developers must write.
- Python supports experimentation and rapid prototyping.
- It can run on Windows, macOS and Linux.

20.2 Important libraries in the course

Library	Main role
NumPy	Numerical computing, arrays, matrices and mathematical operations.
Pandas	Data tables, cleaning, filtering, grouping and preparation.
Matplotlib	Creating plots and visualising data.
SciPy	Scientific computing, optimisation, statistics, integration and linear algebra.
Scikit-learn	Classical machine-learning algorithms for supervised and unsupervised learning.
Theano	A historical numerical-computation and symbolic-expression library used for optimising mathematical computations.
TensorFlow	Machine-learning and deep-learning framework for building and training models.
Keras	High-level neural-network interface that simplifies model construction and training.
PyTorch	Machine-learning framework with tensor operations, automatic differentiation and GPU support.

Version awareness: The source includes older examples and APIs. Some libraries and functions have changed over time. The concepts remain useful, but code should be checked against the current library documentation before running it.

20.3 Python basics

A Python program is made from statements and expressions. Indentation defines code blocks. Variables refer to values, and data types describe what kind of values are stored.

```
Example:
temperature = 28.5
motor_running = True
motor_name = "Pump A"
```

Common data types include strings (text), integers (whole numbers), floating-point numbers (decimal values) and Booleans (True or False).

20.4 Collections

Collection	Description	Example use
List	Ordered, changeable collection.	Store several sensor readings.
Tuple	Ordered, generally immutable collection.	Store a fixed coordinate pair.
Set	Collection of unique items.	Remove duplicate identifiers.
Dictionary	Key-value mapping.	Store a component's properties.
Array	Numerical collection, commonly supplied by libraries such as NumPy.	Perform operations on many measurements.

20.5 Type casting

Type casting converts a value from one data type to another when the conversion is valid. Examples include int("12") to convert text to an integer and float("3.5") to convert text to a decimal number. Some conversions may fail or lose information.

20.6 Operators

Category	Examples	Purpose
Arithmetic	+ - * / // % **	Perform calculations.
Comparison	== != > < >= <=	Compare values; result is Boolean.
Logical	and or not	Combine or reverse conditions.
Bitwise	& | ^ ~ << >>	Operate on binary representations of integers.
Assignment	= += -= *= /=	Assign or update variables.

20.7 Conditional statements

Conditional statements allow a program to choose which instructions to execute. Python uses if, elif and else.

```
if temperature > 80:
    print("Check cooling system")
elif temperature > 60:
    print("Temperature is elevated")
else:
    print("Temperature is within this example's normal range")
```

A nested if places one conditional statement inside another. A conditional expression can place a simple choice on one line. A match-case statement selects among patterns or values.

20.8 Loops

A for loop repeats instructions over items in a sequence or range. A while loop repeats while a condition remains true. The break statement exits a loop, continue skips to the next iteration, and pass is a placeholder that performs no action.

```
for reading in [21, 23, 25]:
    print(reading)

count = 0
while count < 3:
    count += 1
```

20.9 Object-oriented programming

Object-oriented programming (OOP) organises software around objects that combine data and related behaviour. A class defines the structure; an object is an instance of a class.

- **Encapsulation:** keep related data and methods together and control how internal state is accessed.
- **Inheritance:** create a class based on another class, reusing or extending its behaviour.
- **Polymorphism:** allow different object types to respond to the same operation in their own way.
- **Abstraction:** expose essential behaviour while hiding unnecessary implementation detail.
- **Iterators:** provide a way to access items in a sequence one at a time.

20.10 What the library examples demonstrate

The source gives examples of calculating column means with NumPy, filling missing values with Pandas, plotting with Matplotlib, manipulating images with SciPy, classifying the Iris dataset with Scikit-learn, defining mathematical expressions with Theano, building a digit classifier with Keras and training a small network with PyTorch.

These examples illustrate the different roles of the libraries: NumPy handles numerical arrays; Pandas prepares tabular data; Matplotlib visualises it; Scikit-learn supplies conventional ML models; and TensorFlow, Keras and PyTorch support neural-network development.

Code quality warning: Some source examples contain merged lines, obsolete calls or apparent typographical errors. They should not be copied directly into a modern Python environment without checking the relevant library version and repairing the formatting.

21. Quick revision guide

Question	Core answer
What is AI?	Computer systems designed to perform tasks associated with intelligent behaviour.
What are the three traditional CI pillars?	Neural networks, fuzzy systems and evolutionary computation.
What is fuzzy logic?	A way to represent degrees of truth or membership rather than only true/false.
What is the difference between DFS and BFS?	DFS follows a branch deeply; BFS explores level by level.
What does A* evaluate?	f(n) = g(n) + h(n).
What are the three main ML learning settings?	Supervised, unsupervised and reinforcement learning.
What is an MDP?	A framework for sequential decisions using states, actions, transitions and rewards.
What is NLU?	Interpreting the meaning and intent of human language.
What is knowledge representation?	Organising knowledge in a form a computer can reason with.
What is knowledge elicitation?	Obtaining knowledge from experts or other sources.
What is an expert system?	A domain-specific system that applies stored knowledge and reasoning rules.
What is the CIA triad?	Confidentiality, integrity and availability.
What is a distributed system?	Independent networked computers cooperating as one service.
What is pattern recognition?	Identifying meaningful patterns or categories in data.
What is deep learning?	Learning with neural networks that use multiple processing layers.

Useful formulas

- A* evaluation: f(n) = g(n) + h(n).
- Euclidean distance: d(x,y) = √Σ■(x■ – y■)².
- Q-learning: Q(s,a) ← Q(s,a) + α[r + γ max■′Q(s′,a′) – Q(s,a)].
- SARSA: Q(s,a) ← Q(s,a) + α[r + γQ(s′,a′) – Q(s,a)].
- Bellman optimality: V*(s) = max■ Σ■′ P(s′|s,a)[R(s,a,s′) + γV*(s′)].

22. Notes on the supplied source PDF

The supplied PDF contains several formatting and extraction problems. Dates and numbers are missing in some historical passages; some mathematical symbols appear to have disappeared; a few paragraphs are duplicated; and some code examples have been compressed into unreadable lines. The source also includes claims about technologies and library versions that may be time-sensitive.

- Where a historical date or numerical value was missing, it has not been guessed.
- Where an equation could be reconstructed from a standard formula but the source rendering was unclear, the formula is presented as a clearly labelled explanatory form rather than claimed to be a verbatim transcription.
- Some examples are explained conceptually instead of reproducing damaged code.
- Web 4.0 is presented as a vision described by the source, not as a fully completed universal standard.
- The source's discussion of AI and CI recognises multiple views of their relationship; this edition preserves that distinction.
- The source mentions a commercial security vendor and product features. Those vendor-specific details are not expanded here because they are not necessary to understand the core information-security concepts.

Study recommendation: Use this simplified edition alongside the original PDF when checking lecturer-specific wording, diagrams or assessment expectations. If a lecturer provides a corrected formula, date or code sample, use that authoritative correction.

End of GFE 307 Simplified Edition

This document covers the topics identified in the supplied GFE 307 course material. The next course, GFE 306 – Renewable Energy, has not been started.

END